

Ollama



Contents

1 OLLAMA

- 1.1 뭐 LLM은 다 알겠쥬? - 나의 PC에서 돌려보는 LLM
- 1.2 일단 한번 해보자
- 1.3 install
- 1.4 Ollama 간단 사용법
- 1.5 한국어 모델 테스트 해보기
- 1.6 어쨌든 한국 말은 잘한다...
- 1.7 26GB짜리 하나 해보자 험험
- 1.8 NVIDIA GeForce RTX 4060 Ti 8GB 모델
- 1.9 이런 대답을 한다
- 1.10 또 하나 더 허깅페이스
- 1.11 한국어를 잘하는 모델을 가져오자
- 1.12 그리고, modelfile을 하나 만들어두자 -링크-
- 1.13 model make 문법
- 1.14 한글이 되는 LLaMa모델과 modelfile을 한 폴더에 넣어두고,
- 1.15 ollama list를 해서 보면 방금 빌드한 아이가 보인다
- 1.16 잘 동작한다. 빠져 나올때는 /bye 라고 하면 됨
- 1.17 라이브러리 설치

2 이제 OLLAMA와 Python으로 llm 시작

- 2.1 import
- 2.2 우리는 소형 모델에 집중할 생각입니다.
- 2.3 일단 pull
- 2.4 잘 동작하지도 않을 수 있는 고만고만한 모델들
- 2.5 모델별 특징
- 2.6 모델별 성능비교 계획
- 2.7 상식적 배경 이야기 dataclass
- 2.8 dataclass도 한 번 써 봅시다
- 2.9 테스트 결과를 저장할 dataclass
- 2.10 모델 호출 함수
- 2.11 응답속도 측정을 위한 함수

2.12 모델별로 순차적으로 실행

2.13 결과 파싱

2.14 요약 정리

2.15 결과 정리 1

2.16 결과 정리 2

2.17 결과 정리 3 - 실패처리

2.18 지휘자가 필요하겠지?

2.19 뭐, 좀 거창할 순 있지만 -Orchestrator

2.20 predict_main 함수

2.21 temperature라는 옵션

2.22 predict_main 함수 후반 - benchmark 호출

2.23 응답 결과 정리

2.24 결과 - 코딩 능력, 기술적 개념

2.25 언어 번역

2.26 요약

3 소형 모델을 조금 더 써보자

3.1 회의록 정리

3.2 텍스트를 json으로 변환

3.3 결과

3.4 prompt 정의

3.5 프롬프트로 계획 정리 코드

3.6 결과

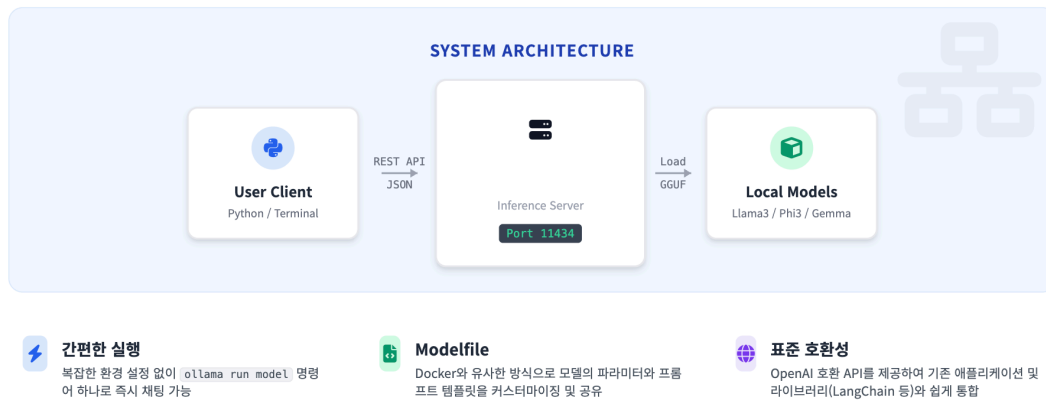
3.7 결과를 마크다운으로

Ollama

Exported from Confluence · <https://pinkwink.atlassian.net>

OLLAMA

뭐 LLM은 다 알겠쥬? - 나의 PC에서 돌려보는 LLM



일단 한번 해보자

<https://ollama.com/> (<https://ollama.com/>)



Get up and running with large language models.

- 디바이스에서 다양한 LLM을 실행/관리 해주는 도구.
- 또한 양자화기능으로 메모리와 계산 효율성을 높일 수 있다.

install

- <https://ollama.com/download/linux> (<https://ollama.com/download/linux>)

Code

```
1 curl -fsSL https://ollama.com/install.sh | sh
```

Download Ollama



Install with one command:

```
curl -fsSL https://ollama.com/install.sh | sh
```

[View script source](#) • [Manual install instructions](#)

Ollama 간단 사용법

- 모델 다운로드 및 실행

Code

```
1 ollama run <model_name>
```

- 모델만 다운로드

Code

```
1 ollama pull <model_name>
```

- 모델 리스트 확인

Code

```
1 ollama list
```

- 모델 삭제

Code

```
1 ollama rm <model_name>
```

한국어 모델 테스트 해보기

Code

```
1 ollama run hf.co/Blossom/llama-3.2-Korean-Blossom-3B-gguf-Q4_K_M:Q4_K_M
```

```
PS C:\Users\hoshi> ollama run hf.co/Blossom/llama-3.2-Korean-Blossom-3B-gguf-Q4_K_M:Q4_K_M
pulling manifest
pulling 389f380bbfbc: 100%
pulling 948af2743fc7: 100%
pulling 6c0b08d96525: 100%
pulling a5943ba50261: 100%
verifying sha256 digest
writing manifest
success
```

어쨌든 한국 말은 잘한다...

>>> 안녕
안녕! 도움이 필요하시면 언제든지 말씀해주세요. 오늘은 어떤 일이 있으세요?

>>> 아이폰
아이폰에 문제가 있을 때는 몇 가지 기본적인 절차를 따르는 것이 중요합니다. 이때 어떤 문제가 있나요? 앱이 안동작하고 있을까요? 비어있게 보이는지, 오류 메시지가 나타나는지, 화면이 비정상적으로 작아지는지 등? 추가 정보를 주시면 더 정확한 도움을 드릴 수 있습니다.

>>> 불국사
불국사는 한국의 역사와 문화에서 중요한 인물입니다. 불국사(不國事)는 5세기 기원후 전반부에 걸쳐 활동한 불교 사상가이자 학자로, 특히 유학을 통해 불교를 전파하고 전설적으로 많은 성적을 이루었다고 알려져 있습니다.

불국사는 주로 한국의 고대 전설과 역사서에서 언급된 인물입니다. 그의 삶은 여전히 많은 연구와 논쟁을 겪고 있으며, 그의 존재는 불교 역사에서 중요한 위치를 차지하고 있습니다. 불국사와 관련된 다양한 이야기와 전설이 한국 문화 속에 깊이 뿌리 내렸습니다.

어떤 부분에 대해 더 알고 싶으시거나, 불국사와 관련된 특정 주제나 질문이 있으신가요?

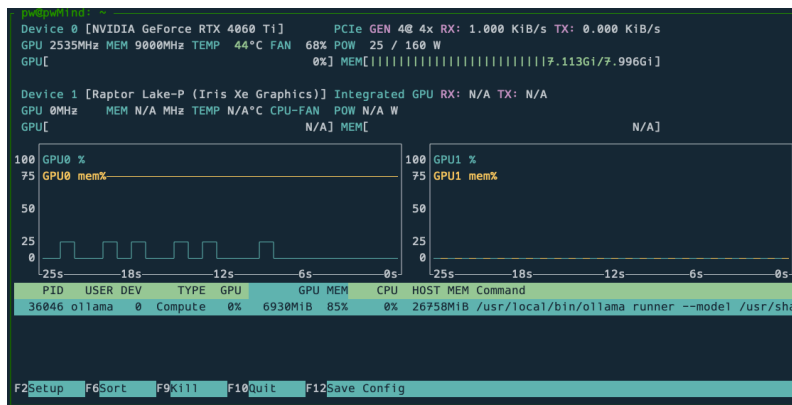
26GB짜리 하나 해보자 험험

Code

```
1 ollama run llama3.3:70b-instruct-q2_K
```

```
pw@pwMind: ~
pw@pwMind:~$ ollama run llama3.3:70b-instruct-q2_K
pulling manifest
pulling 35a6401f84b6: 100%
pulling 948af2743fc7: 100%
pulling bc371a43ce90: 100%
pulling 53a87df39647: 100%
pulling 56bb8bd477a5: 100%
pulling 34ae60f38c8b: 100%
verifying sha256 digest
writing manifest
success
>>> Send a message (/? for help)
```

NVIDIA GeForce RTX 4060 Ti 8GB 모델



이런 대답을 한다

```
>>> 안녕 . IoT 분야의 라즈베리파이보드를 알아?
IoT (인터넷 오브 싱스) 분야에서 Raspberry Pi는 매우 인기 있는 플랫폼입니다 . Raspberry Pi는 ARM 기반의 싱글
보드 컴퓨터로 , 다양한 운영 체제를 지원하며 , Linux와 Python을 포함한 다양한 언어를 지원합니다 .

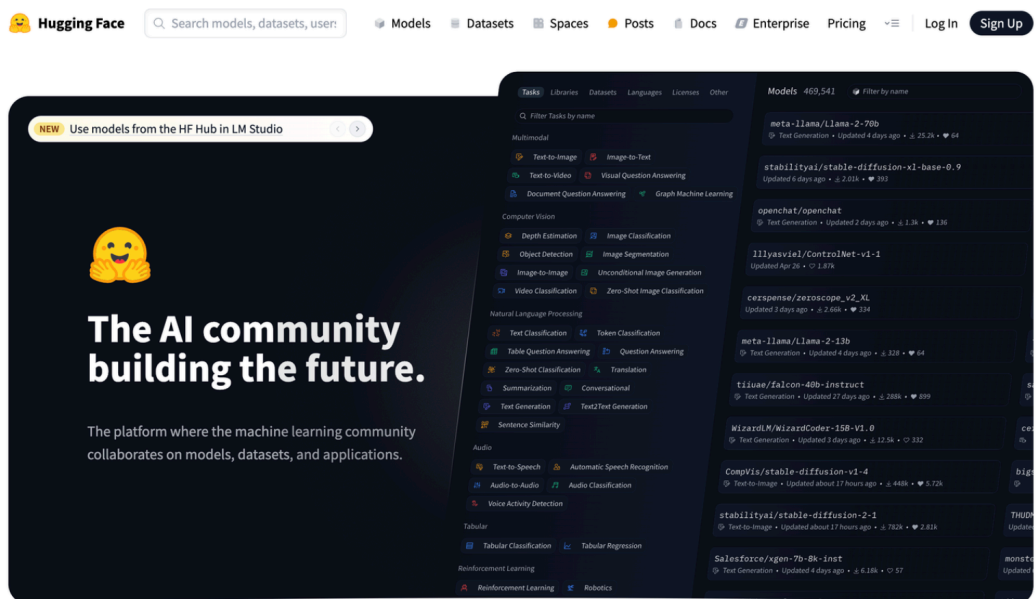
Raspberry Pi는 IoT 프로젝트에 많이 사용됩니다 . 다음과 같은 특징이 있습니다 :

1. 저렴한 가격 : 라즈베리 파이는 매우 저렴한 가격으로 제공되며 , 이는 IoT 프로젝트에 널리 쓰일 수 있는 장점
입니다 .
2. 작은 사이즈 : 라즈베리 파이는 작은 사이즈로 , 다양한 용도로 사용할 수 있습니다 .
3. 다양한 운영 체제 지원 : 라즈베리 파이에서는 다양한 운영 체제를 지원하며 , 이는 개발자들이 자신의 필요에
맞는 운영 체제를 선택할 수 있게 해줍니다 .
4. Python 및 기타 언어 지원 : 라즈베리 파이는 Python을 포함한 다양한 언어를 지원합니다 .

Raspberry Pi는 다음과 같은 IoT 프로젝트에 많이 사용됩니다 :

1. 홈 오토메이션
2. 센서 네트워크
3. 로봇 공학
4. 데이터 수집 및 분석
```

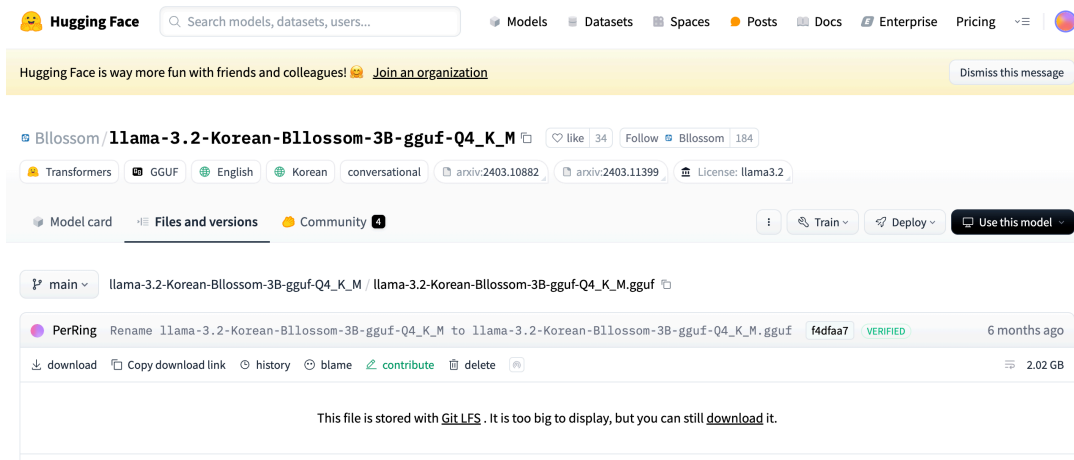
또 하나 더 허깅페이스



- 개발 기관마다 구현 방식이 다른 것이 모델을 활용하는 것에 진입장벽이 생김
- 공통된 인터페이스로 모델을 활용할 수 있도록 지원하는 라이브러리(서비스)가 나타남 - Hugging Face

한국어를 잘하는 모델을 가져오자

- https://huggingface.co/Blossom/llama-3.2-Korean-Blossom-3B-gguf-Q4_K_M/blob/main/llama-3.2-Korean-Blossom-3B-gguf-Q4_K_M.gguf
(https://huggingface.co/Blossom/llama-3.2-Korean-Blossom-3B-gguf-Q4_K_M/blob/main/llama-3.2-Korean-Blossom-3B-gguf-Q4_K_M.gguf)



그리고, modelfile을 하나 만들어두자 - [링크](#)

(https://github.com/PinkWink/for_ROS2_study/blob/main/%EA%B3%B5%EC%A7%9C%EC%88%A0/LLM_study/step1%20modelfile/mode

-

```
Code Blame 15 lines (12 loc) · 326 Bytes
1 FROM ./llama-3.2-Korean-Blossom-3B-gguf-Q4_K_M.gguf
2
3 PARAMETER temperature 0.6
4 PARAMETER top_p 0.9
5 PARAMETER num_ctx 2048
6 PARAMETER stop <s>
7 PARAMETER stop </s>
8
9 TEMPLATE ""{{- if .System }}
10 <s>{{ .System }}</s>
11 {{- end }}
12 <s>Human: {{ .Prompt }}</s>
13 <s>Assistant:"""
14
15 SYSTEM ""You are a helpful and polite AI assistant."""
```

model make 문법

- FROM : 어떤 모델 파일(.gguf)을 쓸지 정함
- PARAMETER : 모델 대답 스타일 설정 (예: temperature 0.6이면 좀 더 차분하게 대답)
- SYSTEM : 모델에게 주는 기본 역할 설명. 예: “넌 친절한 AI야. 잘 도와줘.”
- TEMPLATE : 전체 대화 포맷을 정하는 틀. 유저, 어시스턴트, 도구 메시지를 어떤 형식으로 넣을지 지정
- {{ .System }} 은 위의 SYSTEM 내용이 들어갈 자리
- <|start_header_id|>user<|end_header_id|> 이런 건 대화 순서 구분용 토큰

한글이 되는 LLaMa모델과 modelfile을 한 폴더에 넣어두고,

```
Code
1 ollama create llama3.2-Korean -f modelfile
```

```
~/Downloads/llama model make ollama create llama3.2-Korean -f modelfile
gathering model components
gathering model components gathering model components
copying file sha256:389f380bbf7d9171937b3d166a26ccc3e04201f3d9fb0ca1b5e01b6787621 100%
parsing GGUF
using existing layer sha256:389f380bbf7d9171937b3d166a26ccc3e04201f3d9fb0ca1b5e01b6787621
creating new layer sha256:df9b857dcc013d1a60270394dfdc4d337698607d531de3e8a07386a7f217293
```

ollama list를 해서 보면 방금 빌드한 아이가 보인다

```
~/Downloads/llama model make ollama list
NAME ID SIZE MODIFIED
llama3.2-Korean:latest 6c43a35ffcb0 2.0 GB 10 minutes ago
qwen2.5:1.5b 65ec06548149 986 MB 11 hours ago
~/Downloads/llama model make
~/Downloads/llama model make
```

잘 동작한다. 빠져 나올때는 /bye 라고 하면 됨

```
~/Downloads/llama model make ollama run llama3.2-Korean
>>> 안녕 방가워
안녕, 안녕하세요. 어떻게 도와드릴 수 있나요?
>>> Send a message (/? for help)
```

라이브러리 설치

Code

```
1 pip install langchain-community langchain
2 pip install langchain-ollama
3 pip install ollama
```

이제 OLLAMA와 Python으로 llm 시작

import

표준 라이브러리

시간 측정(`time`), 데이터 구조(`dataclasses`)를 위한 모듈입니다.

Ollama SDK

Python에서 Ollama API와 통신하기 위한 공식 라이브러리입니다.

`pip install ollama`

```
import time
from dataclasses import dataclass
from typing import Dict, Any, List

import ollama # pip install ollama
```

우리는 소형 모델에 집중할 생각입니다.

경량 모델 선정

로컬 PC 환경(GPU/CPU)에서 원활하게 구동 가능한 수십억(B) 파라미터 이하의 소형 모델들을 선정했습니다.

다양한 아키텍처

MS(Phi), Google(Gemma), Alibaba(Qwen), DeepSeek 등 다양한 벤더의 최신 모델을 비교군으로 설정했습니다.

```
MODELS = [
    "phi3:mini",
    "orca-mini:3b",
    "gemma3:4b",
    "qwen3:0.6b",
    "deepseek-r1:1.5b"
]
```

일단 pull

Code

```
1 for model in MODELS:
2     print(f"model: {model} download...")
3     ollama.pull(model)
4     print("completed")
5     print("-" * 20)
```

잘 동작하지도 않을 수 있는 고만고만한 모델들

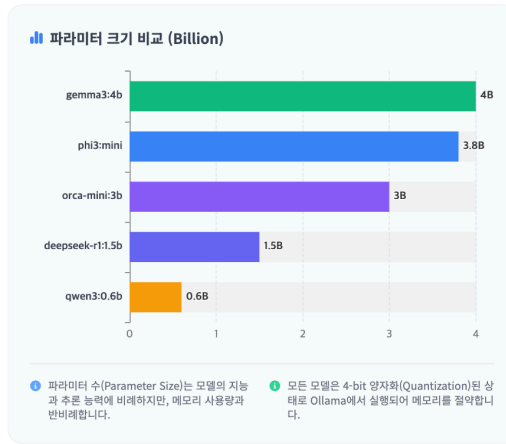
**phi3:mini**
~2.3GB High Perf 3.8B

**gemma3:4b**
~2.5GB Balanced 4.0B

**orca-mini:3b**
~1.9GB Instruct 3.0B

**deepseek-r1:1.5b**
~1.0GB Coding 1.5B

**qwen3:0.6b**
~0.4GB Ultra Fast 0.6B



모델별 특징

**qwen3:0.6b**
Ultra Lightweight

극도로 가벼운 파라미터로 구형 하드웨어나 IoT 기기에서도 즉각적인 응답이 가능합니다.

IoT 제어, 단순 분류, 빠른 응답

**deepseek-r1:1.5b**
Coding Specialist

코드 생성 및 수학적 논리 추론에 특화되어 있으며, 작은 크기 대비 높은 정확도를 보입니다.

Python, 버그 수정, 로직 검증

**orca-mini:3b**
Instruction Follow

복잡한 시스템 프롬프트나 사용자의 지시 사항을 정확하게 따르도록 훈련된 모델입니다.

챗봇 에이전트, 데이터 추출, Q&A

**phi3:mini**
High Performance

동급 크기 대비 가장 뛰어난 성능을 보여주며, 일반적인 상식 추론과 긴 문맥 이해에 강합니다.

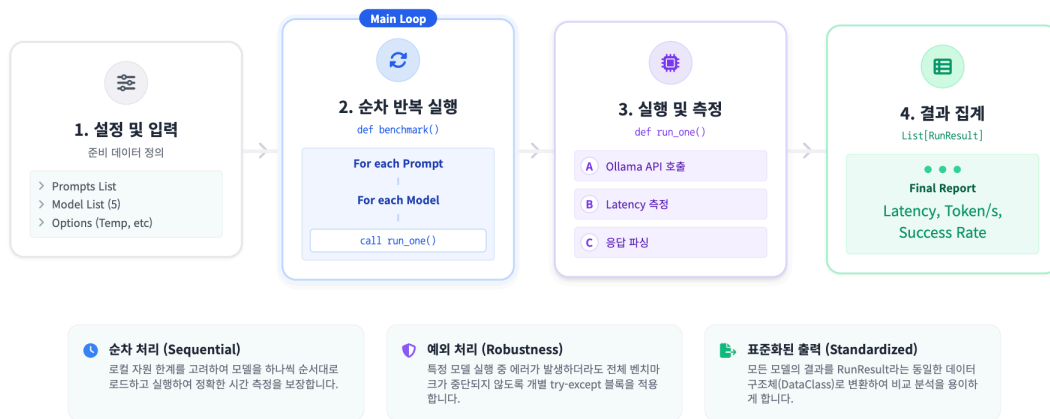
문서 요약, 일반 대화, RAG

**gemma3:4b**
Well Balanced

안정적인 출력과 준수한 성능의 균형을 갖춘 모델로, 다양한 태스크에 범용적으로 활용됩니다.

창의적 글쓰기, 번역, 지식 검색

모델별 성능비교 계획



상식적 배경 이야기 dataclass

TRADITIONAL CLASS
classic_class.py

```

class RunResult:
    def __init__(self, model, ok, latency):
        # 반복적인 할당 코드...
        self.model = model
        self.latency = latency

    def __repr__(self):
        # 디버깅을 위해 직접 구현 필요
        return f"RunResult(model={self.model!r}, "

    def __eq__(self, other):
        # 비교 로직도 직접 구현...
        if not isinstance(other, RunResult):
            return NotImplemented

```

High Boilerplate Lines: 15+

→

WITH @DATACLASS
modern_python.py

```

from dataclasses import dataclass

@dataclass
class RunResult:
    model: str
    ok: bool
    latency: float

# 끝! 나머지는 자동 생성됩니다.

```

Clean & Readable Magic Decorator 자동으로 메서드 주입 Lines: 6

dataclass도 한 번 써 봅시다

```

@dataclass
class RunResult:
    model: str
    latency: float
    ok: bool = True

```

단지 필드만 선언하면 됩니다.
나머지는 Python이 처리합니다.

Auto Generate

자동 생성되는 메서드들

INIT `__init__(self, model: str, ...)`
객체 초기화 메서드. 선언한 필드를 인자로 받아 자동으로 할당합니다.

```
res = RunResult("phi3", 0.5)
```

REPR `__repr__(self) -> str`
디버깅하기 좋은 문자열 표현을 제공합니다.

```
RunResult(model='phi3', latency=0.5, ...)
```

EQ `__eq__(self, other) -> bool`
객체 간의 필드 값을 비교하여 동등성을 확인합니다.

```
res1 == res2 # 필드 값이 같으면 True
```

테스트 결과를 저장할 dataclass

데이터 구조화

벤치마크 실행 결과를 체계적으로 관리하기 위해 dataClass를 정의합니다. 모델명, 지연 시간, 토큰 수 등 핵심 지표를 포함합니다.

타입 안정성

성공(ok=True) 및 실패 케이스를 모두 수용하도록 필드 타입을 Type | None으로 유연하게 설계했습니다.

```

@dataclass
class RunResult:
    model: str
    ok: bool
    latency_s: float
    prompt_eval_count: int | None
    eval_count: int | None
    total_duration_s: float | None
    answer: str | None
    error: str | None

```

모델 호출 함수

메시지 구성

시스템 프롬프트(선택)와 사용자 입력을 OpenAI 호환 포맷의 `list` 형태로 조합합니다.

동기 호출

`ollama.chat` 을 사용하여 모델에 요청을 보내고 전체 응답을 기다립니다.

옵션 전달

`**opts` 를 통해 `temperature`, `num_predict` 등 파라미터를 유연하게 전달합니다.

```
def _sync_chat(model: str, prompt: str, system: str | None = None, **opts):  
  
    messages = []  
    if system:  
        messages.append({"role": "system", "content": system})  
    messages.append({"role": "user", "content": prompt})  
  
    resp = ollama.chat(  
        model=model,  
        messages=messages,  
        options=opts or None,  
    )  
    return resp
```

응답속도 측정을 위한 함수

정밀 시간 측정

`time.perf_counter()` 를 사용하여 CPU 시간을 포함한 시스템 전체 실행 시간을 고해상도로 측정합니다.

응답 + 메트릭 반환

모델 호출 결과인 `resp` 딕셔너리와 측정된 지연 시간(초)을 튜플 형태로 함께 반환합니다.

```
from typing import Dict, Any, Tuple  
  
def call_model_with_timing( model: str, prompt: str,  
    system: str | None = None, **opts ) -> Tuple[Dict[str, Any], float]:  
  
    t0 = time.perf_counter()  
    resp = _sync_chat(model, prompt, system, **opts)  
    t1 = time.perf_counter()  
    latency_s = t1 - t0  
  
    return resp, latency_s
```

모델별로 순차적으로 실행

이중 루프 실행

모든 프롬프트(`prompts`)와 모든 모델(`MODELS`)을 조합하여 순차적으로 테스트를 진행합니다.

결과 집계

각 실행의 결과인 `RunResult` 객체를 리스트에 모아 최종적으로 반환합니다.

`run_one` 호출

```
def benchmark(prompts: List[str], system: str | None = None,  
    **opts) -> List[RunResult]:  
  
    results: List[RunResult] = []  
    for p in prompts:  
        for m in MODELS:  
            print(f"\n>>> Running model={m}")  
            r = run_one(m, p, system, **opts)  
            results.append(r)  
  
    return results
```

결과 파싱

핵심 값 추출

복잡한 JSON 응답에서 비교 분석에 필요한 **답변 본문**과 **토큰 수** (prompt/eval)만 뽑아냅니다.

안전한 접근

모델 버전에 따라 특정 필드가 없을 수 있으므로 `.get()` 메서드를 사용하여 에러를 방지합니다.

단위 변환

Ollama가 반환하는 나노초(ns) 단위의 시간을 읽기 쉬운 초(s) 단위로 변환합니다.

```
def parse_ollama_response(resp: Dict[str, Any]) -> Dict[str, Any]:
    answer = resp.get("message", {}).get("content", "")

    prompt_eval_count = resp.get("prompt_eval_count")
    eval_count = resp.get("eval_count")

    total_duration_ns = resp.get("total_duration")
    total_duration_s = (
        total_duration_ns / 1e9
        if isinstance(total_duration_ns, (int, float)) else None
    )

    return {
        "answer": answer,
        "prompt_eval_count": prompt_eval_count,
        "eval_count": eval_count,
        "total_duration_s": total_duration_s,
    }
```

요약 정리

성공/실패 구분

벤치마크 결과를 순화하며 실행 성공 여부(`r.ok`)에 따라 [OK] 또는 [ERR] 태그를 붙여 출력합니다.

핵심 지표 표시

모델명, 지연 시간(Latency), 프롬프트/생성 토큰 수, 총 소요 시간 등 성능 비교에 필요한 핵심 데이터를 한 줄로 요약합니다.

```
def print_table(results: List[RunResult]):
    print("\n=== RESULTS ===")
    for r in results:
        if r.ok:
            print(
                f"[OK] {r.model:18} "
                f"latency={r.latency_s:.2f}s "
                f"prompt_toks={r.prompt_eval_count} "
                f"gen_toks={r.eval_count} "
                f"ollama_total={r.total_duration_s:.2f}s"
                if r.total_duration_s
                else f"[OK] {r.model:18} latency={r.latency_s:.2f}s"
            )
        else:
            print(f"[ERR] {r.model:18} latency={r.latency_s:.2f}s error={r.error}")
```

결과 정리 1

일관된 인터페이스

성공 여부(`ok`)와 관계없이 항상 동일한 `RunResult` 객체를 반환하여, 호출 측의 처리를 단순화합니다.

명시적 인자 전달

`*` (Asterisk)를 사용하여 모든 파라미터를 키워드 전용(Keyword-Only)으로 강제합니다. 인자 순서 혼동을 방지합니다.

선택적 데이터

`parsed` (성공 데이터)와 `error` (실패 원인)는 상황에 따라 선택적으로 받습니다.

```
def build_run_result(
    *, # 키워드 인자 강제 (Keyword-only arguments)
    model: str,
    ok: bool,
    latency_s: float,
    parsed: Dict[str, Any] | None = None,
    error: Exception | None = None
) -> RunResult:

    # ... (구현 내용은 다음 페이지에서 계속) ...
```

결과 정리 2

성공 케이스 처리

`ok=True` 일 때 실행되는 분기입니다.
파싱된 데이터(`parsed`)가 반드시 존재해야 함을 보장합니다.

데이터 매핑

Ollama 응답에서 추출한 토큰 수, 소요 시간, 답변 내용을 `RunResult` 객체의 필드에 1:1로 매핑하여 반환합니다.

```
if ok:
    assert parsed is not None, "ok=True이면 parsed가 필요합니다."
    return RunResult(
        model=model,
        ok=True,
        latency_s=latency_s,
        prompt_eval_count=parsed.get("prompt_eval_count"),
        eval_count=parsed.get("eval_count"),
        total_duration_s=parsed.get("total_duration_s"),
        answer=parsed.get("answer"),
        error=None,)
```

결과 정리 3 - 실패처리

예외 상황 핸들링

모델 호출 중 에러가 발생했을 때 프로그램이 중단되지 않고 결과를 기록하도록 처리합니다.

데이터 초기화

응답 본문이나 토큰 수 같은 파싱 데이터는 모두 `None` 으로 설정하여 데이터 오염을 방지합니다.

측정값 보존

실패했더라도 시도에 소요된 시간(`latency_s`)과 발생한 에러 메시지를 디버깅을 위해 기록합니다.

```
return RunResult(
    model=model,
    ok=False,           # 실패 플래그 설정
    latency_s=latency_s,
    prompt_eval_count=None,
    eval_count=None,
    total_duration_s=None,
    answer=None,

    error=repr(error) if error else "UnknownError",
)
```

지휘자가 필요하겠지?

🎵 오케스트라의 지휘자처럼

지휘자가 연주자들에게 시작 신호를 보내고, 템포를 조절하며, 실수가 있어도 곡을 완성시키듯, `run_one` 함수는 전체 프로세스를 총괄합니다.



순서 조율 (Coordination)

호출 → 파싱 → 포장의 순서를 엄격하게 관리합니다.



위기 관리 (Control)

예외(Exception)가 발생해도 프로그램이 멈추지 않게 방어합니다.



책임 완수 (Delivery)

성공이든 실패든 항상 표준화된 결과물(`RunResult`)을 보장합니다.

THE ORCHESTRATOR

`run_one()`

모든 책임을 지는 함수



Step 1

API 호출

악기 연주



Step 2

응답 파싱

음정 조율



Step 3

결과 포장

공연 마무리

뭐, 좀 거창할 순 있지만 - Orchestrator

통합 프로세스

앞서 만든 3가지 기능을 하나로 묶어 순차적으로 실행합니다.

1. call_model (호출)
2. parse_response (파싱)
3. build_result (포장)

예외 처리

모델 호출 실패 시 프로그램 종단을 막고, 에러 정보를 RunResult에 담아 반환합니다.

```
def run_one(model: str, prompt: str, system: str | None = None, **opts) -> RunResult:
    try:
        resp, latency_s = call_model_with_timing(model, prompt, system, **opts)
        parsed = parse_ollama_response(resp)
        return build_run_result(
            model=model, ok=True, latency_s=latency_s, parsed=parsed
        )
    except Exception as e:
        latency_s = 0.0
        return build_run_result(
            model=model, ok=False, latency_s=latency_s, error=e
        )
```

predict_main 함수

테스트 프롬프트 구성

다양한 작업 능력을 평가하기 위해 3가지 유형의 질문을 준비합니다.

- 코딩 (버그 수정)
- 기술 설명 (ROS2 개념)
- 자연어 번역 (한영 번역)

공정 비교 옵션

모든 모델이 동일한 조건에서 동작하도록 옵션을 고정합니다.

temperature=0.2 일관된 답변 유도
num_predict=256 생성 길이 제한

```
def predict_main():
    prompts = [
        "다음 파이썬 함수를 버그 없이 고쳐줘:\n\ndef add(a,b):\n return a- b",
        "ROS2에서 nav2 controller_server 역할을 5줄로 설명해줘.",
        "아래 문장을 영어로 자연스럽게 번역해줘: '이번 주에 데모 가능할까요?'"
    ]

    system = "너는 간결하고 정확한 한국어 기술 도우미야. 중간과정은 설명하지마"

    opts = dict(
        temperature=0.2,
        num_predict=256, # 고정 비교 핵심 (생성 토큰 제한)
    )

    # ... (results = benchmark 호출)
```

temperature라는 옵션

Temperature란?

모델이 다음 단어를 선택할 때의 무작위성(Randomness)을 제어하는 값입니다. (범위: 0.0 ~ 1.0)

✓ 왜 0.2를 선택했나?

→ 재현성 확보 (Reproducibility)

같은 질문에 대해 매번 비슷한 답변을 얻기 위함

→ 사실적 답변 (Factuality)

환각(Hallucination)을 줄이고 정확한 정보 전달



Temperature 1.0

창의적, 다양성, 무작위
예: 시 쓰기, 아이디어 발상

Temperature 0.0

결정적, 사실적, 반복 가능
예: 코딩, 데이터 추출, 요약



predict_main 함수 후반 - benchmark 호출

▶ 벤치마크 시작

앞서 설정한 프롬프트 목록과 시스템 메시지, 옵션들을 `benchmark` 함수에 전달하여 실행합니다.

≡ 결과 반환

모든 모델의 순차 실행이 완료되면 수집된 `RunResult` 리스트를 반환합니다.

```
results = benchmark(prompts, system=system, **opts)

return results
```

응답 결과 정리

≡ 결과 순회

저장된 `results` 리스트를 하나씩 가져와 처리합니다.

✓ 성공 여부 확인

`r.ok`가 `True`인 경우에만 구분선, 모델명, 그리고 답변 내용을 출력합니다.

```
for r in results:
    if r.ok:
        print("\n" + "=" * 80)
        print(f"MODEL: {r.model}")
        print(r.answer[:1200])
```

결과 - 코딩 능력, 기술적 개념

 버그 수정 요청

Coding

def add(a,b): return a+ b

phi3:mini / gemma3:4b

✓

return a + b

정확하게 연산자 오류를 찾아 수정함

orca-mini:3b

⚠

"This is not the correct implementation..."

문제는 인식했으나 코드를 직접 수정해서 제시하지 못하고 설명만 시도함

qwen3 / deepseek-r1

✗

No Response (Empty)

 기술 개념 설명

Knowledge

ROS2 nav2 controller_server

gemma3:4b

👍

- 로봇의 위치 결정 및 경로 계획
- 센서 데이터 활용 환경 인식
- 최적 경로 생성 및 제어

가장 정확하고 구조적인 답변 제공

phi3:mini

✗

"colcon clean2 nav2_interfaces_ros..."

존재하지 않는 명령어와 잘못된 맥락(프로젝트 생성 등)을 섞어서 설명함 (환각)

qwen3 / deepseek-r1

✗

No Response (Empty)

언어 번역

번역 능력 차이

간단한 문장 번역에서도 소형 모델 간 성능 편차가 큼니다. **Gemma3**가 가장 정확했으며, 일부 모델은 엉뚱한 대답을 하거나 한영 혼용 (Hallucination)이 발생했습니다.

지시 이행 능력

시스템 프롬프트("중간과정 설명 금지")를 얼마나 잘 따르는지가 핵심입니다. 모델 크기가 작을수록 지시를 무시하고 대화를 시도하는 경향이 있습니다.

[PROMPT] "아래 문장을 영어로 자연스럽게 번역해줘: '이번 주에 데모 가능할까요?'"

MODEL: phi3:mini

"Is it possible to run a demo this month?"

Note: 'this week'가 'this month'로 오역됨 (문맥 파악 오류)

MODEL: gemma3:4b

"Can it be demoed this week?"

Best Result: 의미와 뉘앙스를 정확하게 전달

MODEL: orca-mini:3b

"Sure, I can help you with that. Could you please provide me with..."

Fail: 번역 요청을 수행하지 않고 되물을 (Instruction Following 실패)

MODEL: deepseek-r1:1.5b

"이번 주에 데모가 Possible할 수 있을까요?"

Hallucination: 한국어와 영어가 섞인 부자연스러운 출력

요약

⚡ 속도 (Latency)

소형 모델일수록 응답 속도가 빠름 (환경 의존적)



★ 품질 및 안정성



우수 (Stable)

phi3, gemma3 - 지시 이행 및 한국어 처리 우수



보통 (Unstable)

orca-mini - 문맥 혼란, 때때로 엉뚱한 응답



미흡 (Failed)

qwen3, deepseek-r1 - 응답 누락 빈도 높음

📈 핵심 측정 지표

latency_s

첫 토큰 생성 대기 시간

eval_count

생성된 총 토큰 수

total_duration

전체 수행 시간

🔧 실무 적용 팁

> 다양한 프롬프트 테스트

코딩, 요약, 번역 등 유형별로 모델 성능이 다릅니다.

> 옵션 조정 (Fair Comparison)

num_predict, temperature 등을 고정하여 공정하게 비교하세요.

> 예외 처리 필수

로컬 LLM은 불안정할 수 있으므로 예외/미응답 케이스를 반드시 로깅하세요.

소형 모델을 조금 더 써보자

회의록 정리

```
import ollama
```

```
resp = ollama.chat(  
    model="qwen3:0.6b",  
    messages=[  
        {  
            "role": "system",  
            "content": "너는 한국어를 잘 이해하는 로봇 보조 비서야. 답변은 간결하게 해."  
        },  
        {  
            "role": "user",  
            "content": """"아래 내용을 회의록으로 정리해줘.\n\n다음 주 로봇 데모가 있다\n- 배터리 점검이 필요하다\n- 영상 촬영도 진행한다""""  
        }  
    ],  
    options={  
        "temperature": 0.2, "num_predict": 256}  
)  
  
print(resp["message"]["content"])
```

> EXECUTION RESULT

- 다음 주 로봇 데모 진행
- 배터리 점검 필요
- 영상 촬영 진행

💡 활용 포인트

✓ 시스템 프롬프트의 역할

"답변은 간결하게 해"라는 지시를 통해 불필요한 서론/결론 없이 핵심만 출력

✓ 낮은 Temperature (0.2)

사실적이고 결정적인 요약을 위해 창의성을 제한하여 일관된 결과 확보

✓ 초경량 모델 (0.6B)

단순 요약 작업은 거대 모델 없이도 충분히 수행 가능 (매우 빠른 응답)

텍스트를 json으로 변환

구조화된 출력 요청

자연어로 된 비정형 데이터를 시스템 간 통신에 적합한 JSON 포맷으로 변환하도록 요청합니다.

초경량 모델 활용

qwen3:0.6b와 같은 경량 모델도 명확한 프롬프트가 주어진다면 데이터 변환 작업을 빠르고 정확하게 수행할 수 있습니다.

Prompt Tip

시스템 프롬프트에 "너는 로봇 보조 비서야"와 같은 역할을 부여하고, JSON 형태로 정리해달라는 명시적인 지시를 포함합니다.

```
resp = ollama.chat( model="qwen3:0.6b",
    messages=[
        {
            "role": "system",
            "content": "너는 한국어를 잘 이해하는 로봇 보조 비서야. 답변은 간결하게 해." },
        {
            "role": "user",
            "content": """아래 내용을 JSON 형태로 정리해줘. \n -
                항목: LiDAR 점검, 중요도: 높음 \n-
                항목: 카메라 캘리브레이션, 중요도: 중간 \n-
                항목: 배터리 충전, 중요도: 높음""" } ],
    options={"temperature": 0.2, "num_predict": 256 } )
print(resp["message"]["content"])
```

결과

구조화된 데이터

비정형 텍스트 입력이 JSON Array 형태로 깔끔하게 변환되었습니다.

키-값 매핑

"항목"과 "중요도"가 정확히 키(Key)로 매핑되어 데이터 활용성이 높아졌습니다.

OBSERVATION

qwen3:0.6b 같은 초경량 모델도 명확한 지시사항이 있으면 훌륭한 포매팅 능력을 보여줍니다.

```
[
  {
    "항목": "LiDAR 점검",
    "중요도": "높음" },
  {
    "항목": "카메라 캘리브레이션",
    "중요도": "중간" },
  {
    "항목": "배터리 충전",
    "중요도": "높음" }
]
```

Valid JSON

prompt 정의

목표 설정

단순한 질문이 아닌, 다수의 제약 조건이 포함된 **로봇 데모 운영 계획**을 생성합니다.

주요 제약 조건

- 시간대별 활동 구분 (오전/오후)
- 필수 시연 횟수 지정
- 촬영팀 방문 시간 고정
- 배터리 교체 시간 확보

출력 요구사항

LLM이 결과를 **타임테이블 표**와 **체크리스트** 형태로 구조화하여 답변하도록 명시합니다.

```
# 복잡한 제약 조건을 포함한 프롬프트 정의
prompt = """다음 조건을 만족하는 “로봇 데모 당일 운영 계획”을 만들어줘.

- 오전 10시~12시: 리허설 (자율주행 2회, 팔 조작 2회)
- 오후 1시~3시: 관람객 데모 (자율주행 4회, 팔 조작 3회)
- 각 데모는 12분, 데모 사이 휴식 6분
- 배터리 교체는 15분이 필요하고, 2회 이상 해야 함
- 촬영팀이 2시~2시30분에만 와서 이 시간에는 “팔 조작”이 반드시 1회 포함되어야 함
- 결과는 타임테이블 표 + 체크리스트로 정리해줘
- 그리고 한글로 답해줘
"""
```

프롬프트로 계획 정리 코드

Model: phi3:mini
복잡한 논리적 추론과 긴 형식의 문서 생성을 위해 초경량 모델 대신 **phi3:mini**를 선택했습니다.

Persona 설정
System 메시지로 "실무 엔지니어" 역할을 부여하여, **전문적이고 구조화된 답변**을 유도합니다.

Options 설정
num_predict: 1024: 타임테이블과 체크리스트 출력을 위해 충분한 토큰 길이를 확보합니다.

```
ollama.chat() parameters

resp = ollama.chat(
    model="phi3:mini",
    messages=[
        {
            "role": "system",
            "content": "너는 로봇 데모 운영을 잘하는 실무 엔지니어야. 답은 표와 체크리스트 중심으로"
        },
        {
            "role": "user",
            "content": prompt }
    ],
    options={
        "temperature": 0.3,
        "num_predict": 1024 }
)
print(resp["message"]["content"])
```

결과

로봇 데모 당일 운영 계획:

시간	프로토타인	자율주행	팔 조작	휴식	배터리 교체
10:00 - 12:00 end	자율주행 2회	X			
12:05 - 13:05	자율주행 4회		X	6:00	
13:10 - 13:15				6:00	15:00
13:20 - 13:45	자율주행 2회	X			
13:50 - 14:50	자율주행 2회			6:00	15:00
14:55 - 15:00			X		
15:05 - 17:00 end	자율주행 2회			6:00	15:00
17:05 - 18:00 end	자율주행 2회		X		
18:05 - 19:00 end	자율주행 2회			6:00	15:00
19:05 - 20:00 end	자율주행 4회	X			
20:05 - 20:30			X	6:00	15:00

체크리스트:

- 자율주행: 10시~12시, 13:20~13:45, 17:05~18:0시, 19:05~20:0시, 20:05~20:30
- 팔 조작: 12시10분~12시50분, 14시50분~15시00분, 18시00분~18시30분, 20시00분~20시05분
- 휴식: 13:10~13:15, 15:05~15:30, 17:05~17:45, 19:05~19:45
- 배터리 교체: 13:20~13:35, 18:05~18:20

결과를 마크다운으로

시간	프로토타인	자율주행	팔 조작	휴식	배터리 교체
10:00 - 12:00 end	자율주행 2회	X			
12:05 - 13:05	자율주행 4회		X	6:00	
13:10 - 13:15				6:00	15:00
13:20 - 13:45	자율주행 2회	X			
13:50 - 14:50	자율주행 2회			6:00	15:00
14:55 - 15:00			X		
15:05 - 17:00 end	자율주행 2회			6:00	15:00
17:05 - 18:00 end	자율주행 2회		X		
18:05 - 19:00 end	자율주행 2회			6:00	15:00
19:05 - 20:00 end	자율주행 4회	X			
20:05 - 20:30			X	6:00	15:00



PinkLAB YouTube Channel

PinkLAB Studio — Robotics, AI, ROS2 and more!



Subscribe on YouTube

https://www.youtube.com/@pinklab_studio